

# Mixed Integer Programming

# Contents

- Review –MILP
- Binary Variables
  - Knapsack Example
  - Set Covering Problem Example
  - Purchasing Problem

# Review

# What is a Mixed-Integer Linear Program

- Linear Objective
- Linear Constraints
- Some or all of the decision variables are restricted to be integers

e.g

$$\begin{array}{ll}\text{maximise} & P = 2x + 3.5y \\ \text{subject to:} & \\ & 3x + 2y \leq 14 \\ & 2x + 3y \leq 12.5 \\ \text{and} & \\ & x, y \geq 0 \\ & x, y \in \mathbb{Z}\end{array}$$

$$\begin{array}{ll}\text{maximise} & P = 2x + 3.5y \\ \text{subject to:} & \\ & 3x + 2y \leq 14 \\ & 2x + 3y \leq 12.5 \\ \text{and} & \\ & x, y \geq 0 \\ & x \in \mathbb{Z}\end{array}$$

# Linear Program Solution Without Integers

maximise  $P = 2x + 3.5y$

subject to:

$$3x + 2y \leq 14$$

$$2x + 3y \leq 12.5$$

$$-x + y \leq 1$$

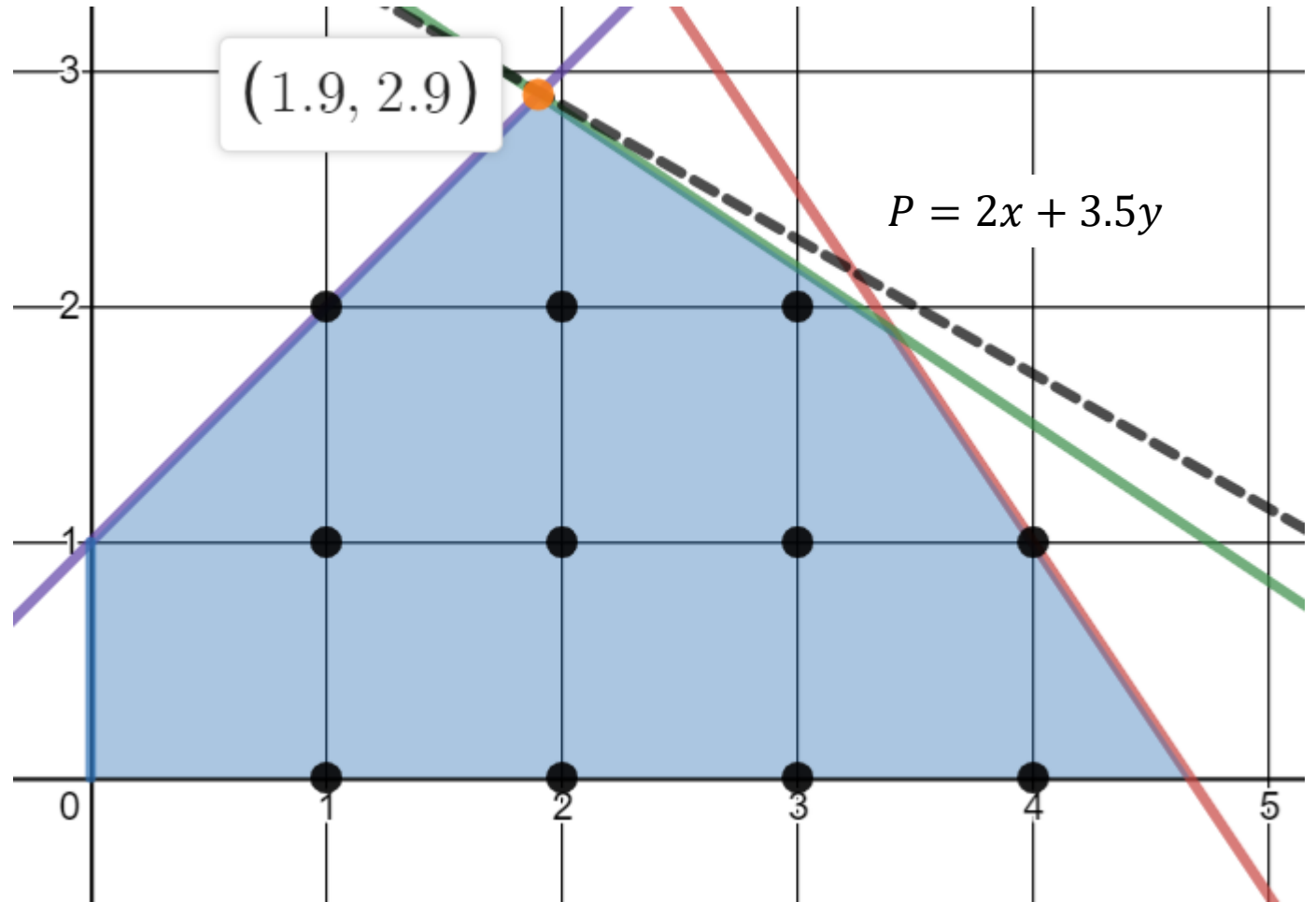
and

$$x, y \geq 0$$

Optimal Solution

$$x^* = (x, y) = (1.9, 2.9)$$

$$P^* = 13.95$$



# Rounded Solutions

## Optimal Solution

$$\mathbf{x}^* = (x, y) = (1.9, 2.9)$$
$$P^* = 13.95$$

$$\mathbf{x} = (x, y) = (1, 2)$$

$$P^* = 9$$

$$\mathbf{x} = (x, y) = (1, 3)$$

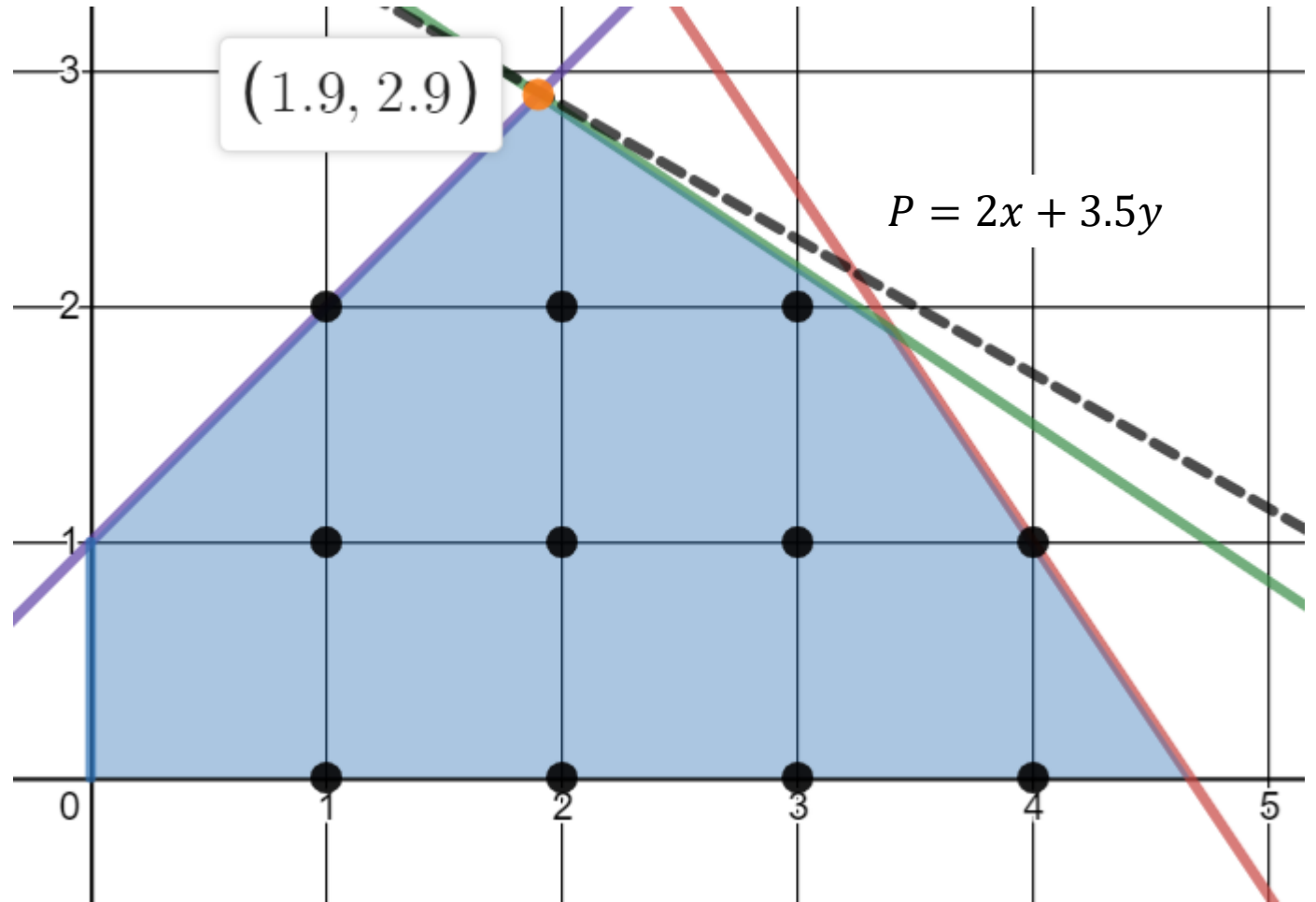
**Infeasible!**

$$\mathbf{x} = (x, y) = (2, 2)$$

$$P^* = 11$$

$$\mathbf{x} = (x, y) = (2, 3)$$

**Infeasible!**



# Rounded Solutions

$$\mathbf{x} = (x, y) = (1, 2)$$

$$P = 9$$

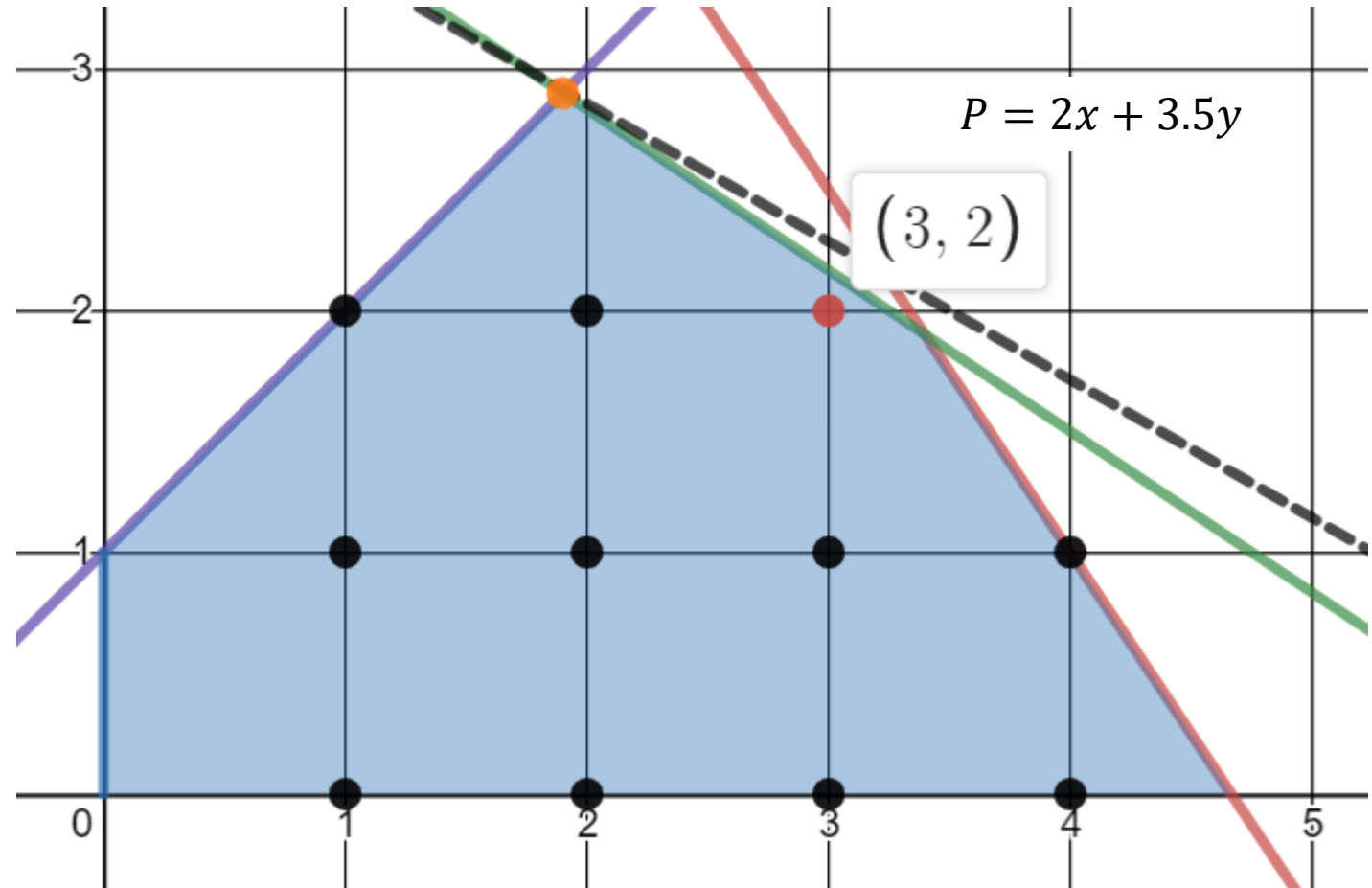
$$\mathbf{x} = (x, y) = (2, 2)$$

$$P = 11$$

However!

$$\mathbf{x}^* = (x, y) = (3, 2)$$

$$P^* = 13$$



# Integer Solution via Pyomo

maximise  $P = 2x + 3.5y$

subject to:

$$3x + 2y \leq 14$$

$$2x + 3y \leq 12.5$$

$$-x + y \leq 1$$

and

$$x, y \geq 0$$

$$x, y \in \mathbb{Z}$$

Optimal Solution

$$\mathbf{x}^* = (x, y) = (3, 2)$$

$$P^* = 13$$

```
1 from pyomo.environ import *
2
3 # Create a concrete model
4 model = ConcreteModel()
5
6 # Define the integer decision variables with lower and upper bounds
7 model.x = Var([0, 1], domain=NonNegativeIntegers)
8
9 # Define the objective function coefficients as a list
10 f = [2, 3.5]
11
12 # Define the objective function
13 model.obj = Objective(expr=sum(f[i] * model.x[i] for i in [0, 1]), sense=maximize)
14
15 # Add the constraints to the model
16 model.c1 = Constraint(expr=3 * model.x[0] + 2 * model.x[1] <= 14)
17 model.c2 = Constraint(expr=2 * model.x[0] + 3 * model.x[1] <= 12.5)
18 model.c3 = Constraint(expr=-model.x[0] + model.x[1] <= 1)
19
20 # Create a solver
21 solver = SolverFactory('glpk')
22
23 # Display the model
24 model.pprint()
25
26 # Solve the model
27 solver.solve(model, tee=False) # Set tee=True to display solver output
28
29 # Display the results
30 model.display()
```

|        |
|--------|
| sol =  |
| 3      |
| 2      |
| fval = |
| -13    |

# Binary Variables

# What is a Binary Variable

- A decision variable that is either 0 or 1

maximise  $P = 2x + 3.5y$

subject to:

$$3x + 2y \leq 14$$

$$2x + 3y \leq 12.5$$

$$-x + y \leq 1$$

and

$$x, y \geq 0$$

$$x \in \{0,1\}$$

maximise  $P = 2x + 3.5y$

subject to:

$$3x + 2y \leq 14$$

$$2x + 3y \leq 12.5$$

$$-x + y \leq 1$$

and

$$x, y \geq 0$$

$$x \leq 1$$

$$x \in \mathbb{Z}$$

This is also an integer variable restricted to  $\geq 0$  and  $\leq 1$

# Knapsack Problem

- 7 items, can only carry 10kg, which to pick to maximise value?



# Knapsack Problem

For  $n$  items and a capacity  $W$

maximise

$$\sum_{i=1}^n v_i x_i$$

subject to:

$$\sum_{i=1}^n w_i x_i \leq W$$
$$x_i \in \{0,1\}, \quad \forall i$$

For our example  $n = 7$  and  $W = 10$

| Item - $x_i$ | Weight (Kg) - $w_i$ | Value (£) - $v_i$ |
|--------------|---------------------|-------------------|
| 1            | 2                   | 10                |
| 2            | 2                   | 10                |
| 3            | 2                   | 10                |
| 4            | 6                   | 100               |
| 5            | 6                   | 100               |
| 6            | 8                   | 130               |
| 7            | 3                   | 70                |

# Knapsack Problem via Pyomo

Maximise

$$\sum_{i=1}^6 v_i x_i$$

subject to:

$$\sum_{i=1}^6 w_i x_i \leq 10$$
$$x_i \in \{0,1\}, \quad \forall i$$

| Item - $x_i$ | Weight (Kg) - $w_i$ | Value (£) - $v_i$ |
|--------------|---------------------|-------------------|
| 1            | 2                   | 10                |
| 2            | 2                   | 10                |
| 3            | 2                   | 10                |
| 4            | 6                   | 100               |
| 5            | 6                   | 100               |
| 6            | 8                   | 130               |
| 7            | 3                   | 70                |

```
1  from pyomo.environ import *
2
3  # Create a concrete model
4  model = ConcreteModel()
5
6  # Define the binary variables
7  model.x = Var(range(7), within=Binary)
8
9  # Define objective function values and weights
10 value = [10, 10, 10, 100, 100, 130, 70]
11 weight = [2, 2, 2, 6, 6, 8, 3]
12 W = 10
13
14 # Define the objective function
15 model.obj = Objective(expr=sum(value[i] * model.x[i] for i in range(7)), sense=maximize)
16
17 # Define the capacity constraint
18 model.capacity = Constraint(expr=sum(weight[i] * model.x[i] for i in range(7)) <= W)
19
20 # Create a solver
21 solver = SolverFactory('glpk')
22 # Display the model
23 model.pprint()
24 # Solve the model
25 solver.solve(model)
26 # Display the results
27 model.display()
```

# Set Covering Problem - Fire Stations Example

- There are six cities in region R. The region must determine where to build fire stations.
- The region wants to build the minimum number of fire stations and ensure that at least one fire station is near 15 minutes of each city.

The times (in minutes) required to drive between cities are:

| City | 1  | 2  | 3  | 4  | 5  | 6  |
|------|----|----|----|----|----|----|
| 1    | 0  | 10 | 20 | 30 | 30 | 20 |
| 2    | 10 | 0  | 25 | 35 | 20 | 10 |
| 3    | 20 | 25 | 0  | 15 | 30 | 20 |
| 4    | 30 | 35 | 15 | 0  | 15 | 25 |
| 5    | 20 | 20 | 30 | 15 | 0  | 14 |
| 6    | 20 | 10 | 20 | 25 | 14 | 0  |

# Set Covering Problem - Fire Stations Example

This is a connected network!

We are removing edges that don't allow us to reach another node in 15 minutes

| City | 1  | 2  | 3  | 4  | 5  | 6  |
|------|----|----|----|----|----|----|
| 1    | 0  | 10 | 20 | 30 | 30 | 20 |
| 2    | 10 | 0  | 25 | 35 | 20 | 10 |
| 3    | 20 | 25 | 0  | 15 | 30 | 20 |
| 4    | 30 | 35 | 15 | 0  | 15 | 25 |
| 5    | 20 | 20 | 30 | 15 | 0  | 14 |
| 6    | 20 | 10 | 20 | 25 | 14 | 0  |

By inspection, build for city 2 and 4.

Covers 1,2,6 and 3,4,5.

# Set Covering Problem - Fire Stations Example

Let

$$x_i = \begin{cases} 1 & \text{if a fire station is built} \\ 0 & \text{otherwise} \end{cases}$$

| City | 1  | 2  | 3  | 4  | 5  | 6  |
|------|----|----|----|----|----|----|
| 1    | 0  | 10 | 20 | 30 | 30 | 20 |
| 2    | 10 | 0  | 25 | 35 | 20 | 10 |
| 3    | 20 | 25 | 0  | 15 | 30 | 20 |
| 4    | 30 | 35 | 15 | 0  | 15 | 25 |
| 5    | 20 | 20 | 30 | 15 | 0  | 14 |
| 6    | 20 | 10 | 20 | 25 | 14 | 0  |

Objective?

Minimise the number of fire stations built!

minimise

$$\sum_i^6 x_i = x_1 + x_2 + x_3 + x_4 + x_5 + x_6$$

# Set Covering Problem - Fire Stations Example

Constraints?

Consider City 1, who can reach it in 15 minutes?

- City 1 and City 2

$$x_1 + x_2 \geq 1 \quad (\text{City 1 Constraint})$$

Consider City 2, who can reach it in 15 minutes?

- City 1, City 2 and City 6

$$x_1 + x_2 + x_6 \geq 1 \quad (\text{City 2 Constraint})$$

| City | 1  | 2  | 3  | 4  | 5  | 6  |
|------|----|----|----|----|----|----|
| 1    | 0  | 10 | 20 | 30 | 30 | 20 |
| 2    | 10 | 0  | 25 | 35 | 20 | 10 |
| 3    | 20 | 25 | 0  | 15 | 30 | 20 |
| 4    | 30 | 35 | 15 | 0  | 15 | 25 |
| 5    | 20 | 20 | 30 | 15 | 0  | 14 |
| 6    | 20 | 10 | 20 | 25 | 14 | 0  |

# Set Covering Problem - Fire Stations Example

minimise

$$\sum_i^6 x_i = x_1 + x_2 + x_3 + x_4 + x_5 + x_6$$

subject to:

$$x_1 + x_2 \geq 1 \quad (\text{City 1 Constraint})$$

$$x_1 + x_2 + x_6 \geq 1 \quad (\text{City 2 Constraint})$$

$$x_3 + x_4 \geq 1 \quad (\text{City 3 Constraint})$$

$$x_3 + x_4 + x_5 \geq 1 \quad (\text{City 4 Constraint})$$

$$x_4 + x_5 + x_6 \geq 1 \quad (\text{City 5 Constraint})$$

$$x_2 + x_5 + x_6 \geq 1 \quad (\text{City 6 Constraint})$$

| City | 1  | 2  | 3  | 4  | 5  | 6  |
|------|----|----|----|----|----|----|
| 1    | 0  | 10 | 20 | 30 | 30 | 20 |
| 2    | 10 | 0  | 25 | 35 | 20 | 10 |
| 3    | 20 | 25 | 0  | 15 | 30 | 20 |
| 4    | 30 | 35 | 15 | 0  | 15 | 25 |
| 5    | 20 | 20 | 30 | 15 | 0  | 14 |
| 6    | 20 | 10 | 20 | 25 | 14 | 0  |

# Set Covering Problem - Fire Stations Example

minimise

$$x_1 + x_2 + x_3 + x_4 + x_5 + x_6$$

subject to:

$$x_1 + x_2 \geq 1$$

$$x_1 + x_2 + x_6 \geq 1$$

$$x_3 + x_4 \geq 1$$

$$x_3 + x_4 + x_5 \geq 1$$

$$x_4 + x_5 + x_6 \geq 1$$

$$x_2 + x_5 + x_6 \geq 1$$

And

$$x_i \in \{0,1\}, \quad \forall i$$

# Set Covering Problem - Fire Stations Example

minimise

$$\sum_i^6 x_i = x_1 + x_2 + x_3 + x_4 + x_5 + x_6$$

subject to:

$$x_1 + x_2 \geq 1 \quad (\text{City 1 Constraint})$$

$$x_1 + x_2 + x_6 \geq 1 \quad (\text{City 2 Constraint})$$

$$x_3 + x_4 \geq 1 \quad (\text{City 3 Constraint})$$

$$x_3 + x_4 + x_5 \geq 1 \quad (\text{City 4 Constraint})$$

$$x_4 + x_5 + x_6 \geq 1 \quad (\text{City 5 Constraint})$$

$$x_2 + x_5 + x_6 \geq 1 \quad (\text{City 6 Constraint})$$

and

$$x_i \in \{0,1\}, \quad \forall i$$

```
1  from pyomo.environ import *
2
3  # Create a concrete model
4  model = ConcreteModel()
5
6  # Define the binary variables
7  model.x = Var(range(6), within=Binary)
8
9  # Define the objective function
10 model.obj = Objective(expr=sum(model.x[i] for i in range(6)), sense=minimize)
11
12 # Define the inequality constraints
13 model.city1 = Constraint(expr=model.x[0] + model.x[1] >= 1)
14 model.city2 = Constraint(expr=model.x[0] + model.x[1] + model.x[5] >= 1)
15 model.city3 = Constraint(expr=model.x[2] + model.x[3] >= 1)
16 model.city4 = Constraint(expr=model.x[2] + model.x[3] + model.x[4] >= 1)
17 model.city5 = Constraint(expr=model.x[3] + model.x[4] + model.x[5] >= 1)
18 model.city6 = Constraint(expr=model.x[1] + model.x[4] + model.x[5] >= 1)
19
20 # Create a solver
21 solver = SolverFactory('glpk')
22 # Display the model
23 model.pprint()
24 # Solve the model
25 solver.solve(model)
26 # Display the results
27 model.display()
```

# Purchasing Problem

Maximise the expected profit given that you have a budget of £160 and can order a maximum of 10 per item.

| Item | Cost (per kg) | Expected Profit (per kg) |
|------|---------------|--------------------------|
| 1    | 10            | 5                        |
| 2    | 12            | 3                        |
| 3    | 9             | 4                        |
| 4    | 5             | 3                        |
| 5    | 13            | 3                        |

In addition, add the following:

- Minimum order placed must be greater than 6
  - You can order at most 2 different items
  - You can order either item 1 or item 4

# Basic Model

Maximum order of 10 per item

Maximise

$$\sum_{i=1}^5 P_i x_i$$

subject to:

$$\sum_{i=1}^5 c_i x_i \leq 160$$

$$\begin{aligned} x_i &\geq 0, & \forall i \\ x_i &\leq 10, & \forall i \end{aligned}$$

| Item | Cost (kg) | Expected Profit (kg) |
|------|-----------|----------------------|
| 1    | 10        | 5                    |
| 2    | 12        | 3                    |
| 3    | 9         | 4                    |
| 4    | 5         | 3                    |
| 5    | 13        | 3                    |

$P_i$  - Profit (per item) for item  $i$

$c_i$  - Cost (per item) for item  $i$

# Updated Problem

Minimum order per item is greater than 6

Maximise

$$\sum_{i=1}^5 P_i x_i$$

subject to:

$$\sum_{i=1}^5 c_i x_i \leq 160$$

$$\begin{aligned} x_i &\geq 6, & \forall i \\ x_i &\leq 10, & \forall i \end{aligned}$$

What is wrong with this?

Answer: I don't have to order an item!

This requires at least 6 of every item to be chosen.

| Item | Cost (g) | Expected Profit (g) |
|------|----------|---------------------|
| 1    | 10       | 10                  |
| 2    | 12       | 12                  |
| 3    | 9        | 9                   |
| 4    | 5        | 5                   |
| 5    | 13       | 13                  |

I want to say the following.



**IF** I choose to order an Item **THEN**

I must order at least 6 and no more than 10

# Binary Variables!

Minimum order per item is greater than 6

Maximise

$$\sum_{i=1}^5 P_i x_i$$

subject to:

$$\sum_{i=1}^5 c_i x_i \leq 160$$

$$x_i \leq 10\delta_i, \quad \forall i \quad (1)$$

$$x_i \geq 6\delta_i, \quad \forall i \quad (2)$$

and

$$\delta_i \in \{0,1\}, \quad \forall i$$

| Item | Cost (g) | Expected Profit (g) |
|------|----------|---------------------|
| 1    | 10       | 10                  |
| 2    | 12       | 12                  |
| 3    | 9        | 9                   |
| 4    | 5        | 5                   |
| 5    | 13       | 13                  |

$\delta_i = 0$  if item  $i$  picked, 1 if item  $i$  not picked

When  $\delta_i = 0$ , constraints (1) and (2) become:

$$x_i \leq 0$$

$$x_i \geq 0$$

i.e. You **don't** choose item  $i$  – force lower and upper bounds of 0

When  $\delta_i = 1$ , constraints (1) and (2) become:

$$x_i \leq 10$$

$$x_i \geq 6$$

i.e. You **do** choose item  $i$  – force lower and upper bounds of  $6 \leq x_i \leq 10$

# In Pyomo

Minimum order per item is greater than 6

Maximise

$$\sum_{i=1}^5 P_i x_i$$

subject to:

$$\sum_{i=1}^5 c_i x_i \leq 160$$

$$x_i \leq 10\delta_i, \quad \forall i$$

$$x_i \geq 6\delta_i, \quad \forall i$$

and

$$\delta_i \in \{0,1\}, \quad \forall i$$

```
1 from pyomo.environ import *
2
3 # Define cost and profit as lists
4 cost = [10, 12, 9, 5, 13]
5 profit = [5, 3, 4, 3, 3]
6
7 # Create a concrete model
8 model = ConcreteModel()
9
10 # Define the decision variables and binary variables
11 model.x = Var(range(5), within=NonNegativeReals)
12 model.b = Var(range(5), within=Binary)
13
14 # Define the objective function
15 model.obj = Objective(expr=sum(profit[i] * model.x[i] for i in range(5)), sense=maximize)
16
17 # Define the constraints
18 model.cost_max = Constraint(expr=sum(cost[i] * model.x[i] for i in range(5)) <= 160)
19 model.select = ConstraintList()
20 model.lower_order = ConstraintList()
21
22 for i in range(5):
23     model.select.add(model.x[i] <= 10 * model.b[i])
24     model.lower_order.add(model.x[i] >= 6 * model.b[i])
25
26 # Create a solver
27 solver = SolverFactory('glpk')
28 # Display the model
29 model.pprint()
30 # Solve the model
31 solver.solve(model, tee=True)
32 # Display the results
33 model.display()
```

# Additional Constraint

You can order amounts of a most 2 items

maximise

$$\sum_{i=1}^5 P_i x_i$$

subject to:

$$\sum_{i=1}^5 c_i x_i \leq 160$$

$$\begin{aligned} x_i &\leq 10\delta_i, & \forall i \\ x_i &\geq 6\delta_i, & \forall i \end{aligned}$$

$$\sum_{i=1}^5 \delta_i \leq 2$$

and

$$\delta_i \in \{0,1\}, \quad \forall i$$

| Item | Cost (g) | Expected Profit (g) |
|------|----------|---------------------|
| 1    | 10       | 10                  |
| 2    | 12       | 12                  |
| 3    | 9        | 9                   |
| 4    | 5        | 5                   |
| 5    | 13       | 13                  |

# In Pyomo

You can order amounts of a most 2 items

Maximise

$$\sum_{i=1}^5 P_i x_i$$

subject to:

$$\sum_{i=1}^5 c_i x_i \leq 160$$

$$\begin{aligned} x_i &\leq 10\delta_i, & \forall i \\ x_i &\geq 6\delta_i, & \forall i \end{aligned}$$

$$\sum_{i=1}^5 \delta_i \leq 2$$

and

$$\delta_i \in \{0,1\}, \quad \forall i$$

```
1  from pyomo.environ import *
2
3  # Define cost and profit as lists
4  cost = [10, 12, 9, 5, 13]
5  profit = [5, 3, 4, 3, 3]
6
7  # Create a concrete model
8  model = ConcreteModel()
9
10 # Define the decision variables and binary variables
11 model.x = Var(range(5), within=NonNegativeReals)
12 model.b = Var(range(5), within=Binary)
13
14 # Define the objective function
15 model.obj = Objective(expr=sum(profit[i] * model.x[i] for i in range(5)), sense=maximize)
16
17 # Define the constraints
18 model.cost_max = Constraint(expr=sum(cost[i] * model.x[i] for i in range(5)) <= 160)
19 model.select = ConstraintList()
20 model.lower_order = ConstraintList()
21 model.no_to_order = Constraint(expr=sum(model.b[i] for i in range(5)) <= 2)
22
23 for i in range(5):
24     model.select.add(model.x[i] <= 10 * model.b[i])
25     model.lower_order.add(model.x[i] >= 6 * model.b[i])
26
27 # Create a solver
28 solver = SolverFactory('glpk')
29 # Display the model
30 model.pprint()
31 # Solve the model
32 solver.solve(model)
33 # Display the results
34 model.display()
```

# Purchasing Problem

You can only order either item 1 or item 4, not both

maximise

$$\sum_{i=1}^5 P_i x_i$$

subject to:

$$\sum_{i=1}^5 c_i x_i \leq 160$$

$$\begin{aligned} x_i &\leq 10\delta_i, & \forall i \\ x_i &\geq 6\delta_i, & \forall i \end{aligned}$$

$$\delta_1 + \delta_4 \leq 1$$

and

$$\delta_i \in \{0,1\}, \quad \forall i$$

| Item | Cost (g) | Expected Profit (g) |
|------|----------|---------------------|
| 1    | 10       | 10                  |
| 2    | 12       | 12                  |
| 3    | 9        | 9                   |
| 4    | 5        | 5                   |
| 5    | 13       | 13                  |

# Purchasing Problem

You can only order either item 1 or item 4, not both

Maximise

$$\sum_{i=1}^5 P_i x_i$$

subject to:

$$\sum_{i=1}^5 c_i x_i \leq 160$$

$$\begin{aligned} x_i &\leq 10\delta_i, & \forall i \\ x_i &\geq 6\delta_i, & \forall i \end{aligned}$$

$$\delta_1 + \delta_4 \leq 1$$

and

$$\delta_i \in \{0,1\}, \quad \forall i$$

```
1 from pyomo.environ import *
2
3 # Define cost and profit as Lists
4 cost = [10, 12, 9, 5, 13]
5 profit = [5, 3, 4, 3, 3]
6
7 # Create a concrete model
8 model = ConcreteModel()
9
10 # Define the decision variables and binary variables
11 model.x = Var(range(5), within=NonNegativeReals)
12 model.b = Var(range(5), within=Binary)
13
14 # Define the objective function
15 model.obj = Objective(expr=sum(profit[i] * model.x[i] for i in range(5)), sense=maximize)
16
17 # Define the constraints
18 model.cost_max = Constraint(expr=sum(cost[i] * model.x[i] for i in range(5)) <= 160)
19 model.select = ConstraintList()
20 model.lower_order = ConstraintList()
21 model.item1_or_item4 = Constraint(expr=model.b[0] + model.b[3] <= 1)
22
23 for i in range(5):
24     model.select.add(model.x[i] <= 10 * model.b[i])
25     model.lower_order.add(model.x[i] >= 6 * model.b[i])
26
27 # Create a solver
28 solver = SolverFactory('glpk')
29 # Display the model
30 model.pprint()
31 # Solve the model
32 solver.solve(model)
33 # Display the results
34 model.display()
```